

Code review — academy-hub-api

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 6 — 2026-10-01 — sha `bbfcd9b`

Scop: starea celor 9  din runda 5, după două commituri (`22a4521` la 1h39m de la livrarea reviewului, `bbfcd9b` a doua zi).

Proiectul **compilează** (`mvn -DskipTests compile` ) și pornește pe o bază nouă. Toate constatările au fost reproduse prin rulare: copie în afara repo-ului, H2 în loc de MySQL, portul 8095, **~45 de cereri HTTP reale**, plus două porniri succesive pe un H2 **pe fișier** (ca să probez din nou B4) și două metode de listare expuse temporar printr-un controller-sondă **doar în copie** (ca să probez B3). Codul din repo nu a fost modificat.

Ce s-a închis — șase dintr-o rundă, cel mai bun raport de până acum

- **B5** `orElseThrow()` **gol** (deschis la runde 4 și 5) — **închis complet**. Zero apariții de `orElseThrow()` fără argument în tot `src/`. `POST /api/books` cu `studentId` inexistent → **404 „Student not found”** (era 500). Toate cele 7 locuri, nu doar cel semnalat — prima dată când duci o regulă până la capăt.
- **B2** `PUT /api/students/{id}` — **închis**, probat pe toate trei cazurile:

```
PUT cu ACELASI email, varsta 32    -> 200    (era 409)
PUT cu emailul Mariei              -> 409    Email already used
PUT cu un email NOU                -> 200
```

- **B3** `GET /api/enrollments/{id}` **inexistent** → **404** (era 500 prin NPE). `getId` scos din `EnrollmentRepository`, înlocuit cu `findById(...).orElseThrow(...)`.
- **B6 duplicatul de carte** (deschis la runde 4 și 5) — **închis**. `existsByStudentIdAndName(studentId, name)` în `BookRepository:25`. Probat: `POST /books {Effective Java, Maria}` → **200** (cartea e a lui Rareș, nu a ei).
- **B7** `PUT` **pe o înscriere cu aceeași pereche** (deschis de **trei** runde) — **închis**: `PUT /enrollments/{e1}` cu perechea pe care o are deja → **200**. Rândul nu se mai declară propriul duplicat.
- **B4 listele goale** (deschis de **trei** runde) — **închis pe tot ce e expus**:

```
GET /api/students      pe baza goala -> 200  []
GET /api/courses       pe baza goala -> 200  []
GET /api/books         pe baza goala -> 200  []
GET /api/enrollments   pe baza goala -> 200  []
```

14 din 16 metode de listare curățate. Restul: B3 de mai jos.

- **B1, jumătatea „not found” — închisă.** `NoCourseFound` înregistrată, `CourseIdNotFound` / `StudentIdNotFound` șterse. Probat:

```
GET /api/courses/{inexistent}    -> 404  No course found
GET /api/students/{inexistent}   -> 404  Student not found
GET /api/books/{inexistent}      -> 404  Book not found
GET /api/enrollments/{inexistent} -> 404  Enrollment not found
```

Patru resurse, același „nu există”, **același răspuns** — exact ce lipsea la runda 5.

- **M1 excepțiile duble** (runda 5) — **închis**. Ai șters `CourseIdNotFound` , `StudentIdNotFound` , `TextArgumentNumberInvalid` , `BookAlreadyExists` . Plus `StudentFactory` , `StudentFactoryImpl` , `StudentAgeComparator` — cod mort, 250 de linii scoase.
- **M2 `findByDepartmentOrderByNameAsc`** sortează acum efectiv (`CourseQueryServiceImpl:54`).
- **M12 două adnotări `@Transactional`** — **închis**: toate patru command service-urile folosesc `org.springframework.transaction.annotation.Transactional` .
- **M13 `@PathVariable` în interfața de service** — **închis**.
- **C1 importuri nefolosite**: de la 22 în 19 fișiere la **14 în 11**.
- **Swagger documentat**: `@ApiOperation` + `@ApiResponses` pe `BookController` , cu `ApiError` ca schemă de eroare. E începutul bun; vezi M11.

Critice

B1 — Jumătatea „conflict” a registrului: 5 locuri dau tot 500 (runda 5, B1)

`sistem/GlobalExceptionHandler.java:31-33` și `:50`

CUM E ACUM: ai reparat familia „not found” — complet, toate patru resursele. Familia „conflict” e neatinsă: în advice e înregistrată **o singură** excepție de conflict, `EmailAlreadyUsed` . Reprodus azi:

```
POST /courses nume duplicat (Java Fundamentals)      -> 500
POST /enrollments pereche existenta (Rares+Java)      -> 500
POST /books carte deja la Maria (Head First Java)    -> 500
PUT /courses/{Spring Boot} name='Java Fundamentals' -> 500
PUT /enrollments/{e1} perechea Rares+Spring          -> 500
```

Cele cinci locuri, pe nume:

NEÎNREGISTRATE, dar aruncate:

```
CourseNameAlreadyInUse          2 locuri
  CourseCommandServiceImpl:29   (create)
  CourseCommandServiceImpl:56   (update)
StudentAlreadyEnrolledInThisCourse 2 locuri
  EnrollmentCommandServiceImpl:39 (create)
  EnrollmentCommandServiceImpl:74 (update)
BookAlreadyAssignedToThisStudent 1 loc
  BookCommandServiceImpl:40
                                     5 locuri
```

ÎNREGISTRATĂ, dar niciodată aruncată:

```
EmailNotFound                   0 locuri
```

DE CE: mecanismul e cel explicat la runda 5 și el s-a confirmat exact: un `@RestControllerAdvice` e un **registru pe tip**. Dovada o ai în corpul răspunsului — uită-te la diferența dintre cele două familii:

```
404: {"timestamp":"...", "status":404, "error":"Not Found",
      "message":"No course found", "path":"...", "details":{}}

500: {"timestamp":"...", "status":500, "error":"Internal Server Error",
      "path":"/api/courses"}
```

Al doilea corp **nu are message**, **nu are details**. Nu e `ApiError` -ul tău: e corpul scris de `BasicErrorController` după ce cererea a fost redirectată spre `/error`. Când vezi un răspuns de eroare fără `message`, știi fără să te uiți în cod că advice-ul tău n-a fost chemat deloc.

Partea care contează pentru învățare nu e mecanismul — pe ăla l-ai aplicat corect la not-found. E **unde te-ai oprit**. Review-ul rundeii 5 îți dădea fixul în două blocuri, unul lângă altul: `handleNotFound` cu 6 clase și `handleConflict` cu 4. Ai scris primul, nu pe al doilea. Aceeași greșală de formă ca la „repară linia, nu regula”, doar că aici linia e o **listă**: ai completat lista pe care o aveai deja și n-ai creat-o pe a doua.

Și mai e dovada de ce nu ai văzut-o singur: **colecția Postman e neatinsă în ambele commituri**, tot 28 de cereri. În folderul `errori/` ai patru cereri de 400, două de 404 și două de 409 — ambele de 409 pe `POST /api/students`, singura excepție de conflict înregistrată. Nicio cerere cu nume de curs duplicat. Nicio pereche

de înscriere retrimisă. Nicio carte trimisă a doua oară aceluiași student. Regula de la runda 5 rămâne aceeași, și acum ai dovada că funcționează: **pentru fiecare** `throw new` **din** `src/`, **o cerere care îl declanșează**. Ai 10 `throw new` și 5 cereri care ating 5 dintre ele.

FIX: al doilea bloc, cel pe care nu l-ai scris:

```
@ExceptionHandler({EmailAlreadyUsed.class, CourseNameAlreadyInUse.class,
    StudentAlreadyEnrolledInThisCourse.class,
    BookAlreadyAssignedToThisStudent.class})
public ResponseEntity<ApiError> handleConflict(RuntimeException exception,
    HttpServletRequest request) {
    return build(HttpStatus.CONFLICT, exception.getMessage(),
        request, List.of());
}
```

`EmailNotFound` se scoate din `handleParticularNotFound`: nu e aruncată de nimeni.

B2 — REGRESIE: `PUT /api/courses/{id}` a trecut de la 409 la 500

`course/services/CourseCommandServiceImpl.java:54-56` · `sistem/GlobalExceptionHandler.java:104`

CUM E ACUM: aceeași cerere, două runde:

```
runda 5: PUT /courses/{id} cu numele altui curs
-> 409 {"message":"Request violates a database constraint"}

runda 6: PUT /courses/{id} cu numele altui curs
-> 500 {"status":500,"error":"Internal Server Error"}
```

Verificarea pe care ți-am cerut-o la B9 **e scrisă și funcționează** — se vede chiar din asta că funcționează. Numai că răspunsul pe care îl primește clientul e acum mai rău decât înainte.

DE CE: la runda 5 cererea trecea prin `updateCourse` fără verificare, ajungea la `flush`, iar baza refuza scrierea cu `DataIntegrityViolationException` — o excepție **Spring**, care era în registrul tău (`:104`), cu 409 pe ea. Statusul venea de la plasa de siguranță, nu de la regula ta.

Acum regula ta se execută prima, aruncă `CourseNameAlreadyInUse`, iar cererea nu mai ajunge niciodată la bază — deci nici la plasă. Și `CourseNameAlreadyInUse` nu e în registru. Ai mutat decizia de la baza de date în codul tău, care e corect, dar jumătatea a doua a mutării — să spui advice-ului ce să facă cu excepția nouă — n-a fost făcută.

Lucrul de învățat aici e mai general decât cele două linii: **când muți o verificare mai devreme pe drumul cererii, schimbi și excepția care iese**. Un fix care înlocuiește o excepție cu alta nu e gata când condiția e corectă; e gata când răspunsul HTTP e corect. De-aia a doua coloană din tabelul Before/After de la runda 5 nu era doar codul, ci și statusul așteptat.

Și observă ce ți-a ascuns asta: dacă te-ai fi uitat **numai** la status, ai fi zis că runda 5 era mai bună. Dacă te uiți la **message**, vezi că la runda 5 regula ta nu rula deloc. Ambele runde erau greșite, în locuri diferite. Statusul singur nu e o probă — corpul răspunsului e.

FIX: se închide odată cu B1 — **CourseNameAlreadyInUse** în **handleConflict**. Niciun cod nou în **CourseCommandServiceImpl**; partea de acolo e corectă.

Cum verifici: **PUT /courses/{id}** cu numele altui curs → **409** cu **"message": "Course name already in use"** — mesajul **tău**, nu cel al bazei. **PUT** cu numele propriu, nemodificat → 200.

B3 — Două metode de listare au rămas cu **throw** (runda 5, B4)

student/services/StudentQueryServiceImpl.java:44 · **:87**

CUM E ACUM: 14 din 16 metode de listare au fost curățate. Două nu. Pentru că niciuna nu e expusă prin HTTP, le-am expus temporar **în copia mea**, printr-un controller-sondă, ca să vezi exact ce se întâmplă în ziua în care le expui tu:

cu date în baza:

```
/proba/age-less-than-100    -> 200  [...]
/proba/students-with-books   -> 200  [...]
```

pe baza GOALA:

```
/proba/age-less-than-100    -> 404  "Student not found"    SQS:44
/proba/students-with-books   -> 500                               SQS:87
/proba/book-counts          -> 200  []                <- reparata de tine
```

DE CE: cele două linii rămase arată de ce „am reparat lista goală” nu e același lucru cu „am aplicat regula”. Regula e: *o listă nu poate lipsi, poate doar să fie goală*. Ai aplicat-o în 14 locuri — inclusiv în toate cele pe care le atingeai un endpoint. Cele două rămase sunt exact cele pe care nu le-a atins nicio cerere Postman.

Și uită-te cum arată consecințele, pentru că sunt **diferite** și amândouă greșite:

- :44** aruncă **StudentNotFound**, care **e** în registru → iese un **404** cu „Student not found”. Un 404 arată ca un răspuns îngrijit, deci asta e cazul periculos: cineva care cere „studentii sub 100 de ani” pe o bază goală primește „studentul nu a fost găsit” — un mesaj despre un singur student, la o întrebare despre o mulțime. Nu e o eroare de status, e o eroare de **sens**: 404 spune „resursa de la adresa asta nu există”, iar lista există, doar că e goală.
- :87** aruncă **NoStudentsFound**, care **nu** e în registru → **500**.

Tiparul de fond e cel de la B1: ai dus regula până unde ajungea o cerere pe care o aveai deja scrisă. Ce nu e acoperit de o cerere rămâne în starea în care era.

FIX: aceeași formă ca în celelalte 14:

```

@Override
public List<Student> getStudentsWithAgeLessThan(int age) {
    return studentRepository.findAllByAgeLessThan(age);
}

@Override
public List<Student> findAllStudentsWithBooks() {
    return studentRepository.findAllStudentsWithBooks();
}

```

Apoi `NoStudentsFound`, `NoBookFound`, `NoEnrollmentsFound` nu mai sunt aruncate de nimeni — se șterg, împreună cu constantele lor.

B4 — Fiecare pornire șterge tot ce a scris API-ul — a treia rundă, linia neatinsă

`Runner.java:78-79` · `:85-90` · `:106-109`

CUM E ACUM: reprodus din nou, identic cu runda 5, pe H2 **pe fișier**:

```

pornire 1: 8 studenti din seed
           + POST /api/students ion.pop@test.ro    -> 200
           + POST /api/students ana.ene@test.ro    -> 200
           10 studenti in baza

--- opresc si repornesc aplicatia, aceeasi baza ---
pornire 2: 8 studenti
           supravietuitori test.ro: []

```

DE CE: `Runner` e `@Component` și implementează `CommandLineRunner`, deci `run()` nu e un script pe care îl pornești când vrei — e o metodă pe care Spring o cheamă **la fiecare pornire**. Înăuntru, `reset()` (`:85-90`) face `deleteAllInBatch()` pe toate patru tabelele, iar `seed()` mai face pe deasupra încă un rând de `deleteAll()` (`:106-109`): ștergerea se întâmplă de două ori, din două locuri.

Constatarea e deschisă de trei runde (4 B2 · 5 B8) și e singura dintre cele nouă pe care n-ai atins-o deloc. Merită spus de ce insist: cât timp `Runner` rescrie seed-ul la fiecare pornire, nu ai cum să-ți probezi propriile conflicte. Cele cinci cereri de la B1 cer stări pe care le construiești tu — un nume de curs ocupat, o pereche deja înscrisă, o carte deja atribuită. Dacă baza revine la seed la fiecare `Ctrl+R` din IntelliJ, construirea stării începe de la zero de fiecare dată și renunți la ea.

Altfel spus: B4 nu e doar o pierdere de date, e **motivul pentru care B1 a supraviețuit**. Cele două se repară în ordinea asta: mai întâi `Runner`, ca să ai pe ce lucra.

FIX: `@Profile("seed")` pe clasa `Runner`, pornit explicit doar când vrei date de test:

```
@Component
@Profile("seed")
public class Runner implements CommandLineRunner { ... }
```

Apoi `mvn spring-boot:run -Dspring-boot.run.profiles=seed` când vrei seed, și pornirea obișnuită când nu vrei. Varianta minimă, dacă vrei doar să nu mai pierzi date azi: comentezi `reset()` și `seed()` la `:78-79` și ștergi blocul `deleteAll()` din `seed()`.

🟡 Importante

M1 — interogarea rulează de două ori, în 5 locuri (de la 13). `StudentQueryServiceImpl.java:32+35`, `:44+47`, `:63+66`, `:71+74`, `:87+90`. Tiparul e „întreb dacă e goală, apoi o cer din nou”: două `SELECT` identice pentru un răspuns. Trei dintre ele dispar odată cu B3. Semnalat la runde 2, 3, 4 și 5.

M2 — `Optional` care aruncă în loc să fie gol — și `orElseThrow` mort la apelant.

`StudentQueryServiceImpl.java:62-67` și `:70-75`, `CourseQueryServiceImpl.java:66-68`. Cea din urmă e cea mai clară: `Optional.ofNullable(findById(id).orElseThrow(...))` — `orElseThrow` garantează că valoarea nu e `null`, deci `ofNullable` nu are ce să ambaleze. Consecința e la apelant: în `EnrollmentCommandServiceImpl:76-77`, `.orElseThrow(NoCourseFound::new)` și `.orElseThrow(StudentNotFound::new)` nu se pot executa niciodată, pentru că metodele au aruncat deja înăuntru. Un tip `Optional` e un contract: „poate să nu fie nimic aici, și apelantul decide”. Dacă metoda aruncă, întoarce `Student`, nu `Optional<Student>`. Semnalat ca M4 la runde 2, 3, 4 și 5.

M3 — `POST` răspunde 200, nu 201, pe toate patru modulele. `StudentController:45`.

`CourseController:39` · `BookController:56` · `EnrollmentController:38`. Verificat pe antete azi: `HTTP/1.1 200`, fără `Location`. Semnalat la runde 3, 4 și 5. Acum că ai `GET /{id}` pe toate patru, `Location` e o linie.

M4 — `DELETE /api/enrollments` ia id-urile din corp; `DELETE /{id}` dă 405. Probat: `DELETE /api/enrollments/{id}` → `405`, și răspunsul de 405 nici nu trece prin `ApiError` (n-are `message`). `EnrollmentController:48`. Semnalat la runde 3, 4 și 5.

M5 — `PUT /api/books/{id}` cere `createdAt` obligatoriu și îl suprascrive. `BookUpdateRequest:12` + `BookCommandServiceImpl:77`. Data la care un rând a fost creat nu e un câmp editabil (runde 4 și 5).

M6 — ștergerea unui curs șterge în tăcere înscrierile. Reprodus azi: `DELETE /api/courses/{Java Fundamentals}` → 200; înscrierile scad de la **18 la 13**, iar răspunsul nu spune nimic despre cele 5. Din `cascade = CascadeType.ALL` + `orphanRemoval = true` pe `Course.enrollments`. Alternativa uzuală: refuzi ștergerea cât timp există înscrieri (409) sau marchezi cursul inactiv (runde 4 și 5).

M7 — 19 din 27 de metode de query tot nu sunt expuse — și atenție CUM le expui. Cele 9 căi din OpenAPI sunt neschimbate. Când le expui, două capcane, amândouă măsurate azi pe sonda mea: - `throw`-ul de la B3 (reparat în 14 metode, rămas în 2); - metodele care întorc **entități**, nu DTO-uri. Am expus `findAllStudentsWithBooks()` așa cum e și am comparat cu `GET /api/students`, care trece prin

`StudentResponse` : 1 717 baiți (DTO) vs **20 938** baiți (entitate), cu cuvântul `"student"` apărând **166 de ori** în răspuns — pentru că `Student` are `books`, iar fiecare `Book` are `student`, care are `books` ... Legătura bidirecțională din JPA devine, la serializare, text repetat. DTO-ul nu e o formalitate: el e locul în care tai legătura.

M8 — structura cerută la runde 1, 2, 3, 4 și 5, neatinsă. `services/interfaces/` în loc de `services/` + `services/impl/` · plural inconsecvent (`models`, `dtos`, `services`, `controllers`, dar `repository` la singular) · `ExceptionConstants` în patru pachete cu același nume. Nou: pachetul `web/` a devenit `sistem/` — un nume **românesc** într-un cod scris integral în engleză (`academy.hub.app.sistem.GlobalExceptionHandler`). Numele pachetelor fac parte din API-ul codului; alege engleza, ca restul. `web/` sau `api/` era deja bine.

M9 — trei convenții de nume în același modul. `EnrollmentCreateResponse` întoarce `enrollmentId`, `studentId`, `courseId`; `EnrollmentDeleteResponse` întoarce `enrollmentID`, `studentID`, `courseID`; `EnrollmentUpdateResponse` întoarce `id`, `studentId`, `courseId`. Neatins (runda 4, M11 · runda 5, M11). `ApiError` arată cum se face: o formă, aceeași peste tot.

M10 — `handleParamValidation` e cod mort. `GlobalExceptionHandler:67–79` prinde `HandlerMethodValidationException`. Două motive pentru care nu se execută niciodată: (1) niciun parametru de metodă de controller nu are vreo constrângere (`@NotBlank`, `@Min`, `@Size`) — n-are ce să eșueze; (2) cele patru controllere au `@Validated` pe clasă, iar `@Validated` validează prin proxy AOP și aruncă `ConstraintViolationException`, nu `HandlerMethodValidationException` — deci chiar dacă ai pune o constrângere, ar ajunge la `handleParameterValidation` (:55), nu la asta. Alege: sau scoți `@Validated` de pe controllere și păstrezi handlerul, sau îl păstrezi și ștergi handlerul. Acum le ai pe amândouă și una e inertă. Nu e un bug — e o linie pe care crezi că te poți baza și nu poți.

M11 — Swagger documentat pe un controller din patru, 3 metode din 5. `BookController` are `@Operation` + `@ApiResponses` pe `getBooks`, `createBook`, `deleteBook`; `getBookById` și `updateBook` nu. Celelalte trei controllere, deloc. Bun că ai pornit și bun că ai legat `ApiError` ca schemă de eroare — dar o documentație parțială e mai înșelătoare decât niciuna: cine se uită în `/swagger-ui.html` crede că `PUT /api/books/{id}` nu poate da 404.

M12 — `DataIntegrityViolationException` → 409 pentru tot. :104. Clasa acoperă și `NOT NULL`, și cheia străină, și lungimea coloanei — lucruri care nu sunt conflicte, ci cereri invalide (400) sau defecte ale tale (500). Odată ce B1 e închis, plasa asta ar trebui să rămână ca ultimă soluție, nu ca sursă de 409-uri (runda 5, B9).

● Cleanups

- C1 — 14 importuri nefolosite**, în 11 fișiere (de la 22 în 19): `BookQueryServiceImpl:5` (`NoBookFound`), :9 (`Component`), :12 (`ArrayList`) · `EnrollmentQueryServiceImpl:5` (`NoEnrollmentsFound`), :6 (`Enrollment`) · `Course.java:7` (`NotNull`) · `CourseSummary:5` (`Function`) · `CourseCommandService:4` (`CourseRepository`) · `EnrollmentUpdateResponse:3` · `EnrollmentQueryService:4` · `StudentRepository:6` (`NotBlank`) · `StudentUpdateRequest:5` (`UUID`) · `StudentQueryService:7` (`Order`) · `BookRepository:4` (`Valid`). Tiparul e tot cel de la runda 5: repara o linie, importul vechi rămâne. `Ctrl+Alt+0` în IntelliJ.

- **C2** — `deletebook` , `updatebook` cu literă mică, în interfață și în implementare. Semnalat la runde 2, 3, 4 și 5.
 - **C3** — `EnrollmentController:27` : metoda care întoarce o înscriere se numește `getCourseById` . Copiată din `CourseController` și neredenumită (runda 5, C3).
 - **C4** — `Runner.java` are **671 de linii** (de la 718), majoritatea metode de test comentate: `scsTEST` , `bcsTEST` , `cqsTest` , `eqsTEST` sunt corpuri aproape goale. `ecsTEST` are UUID-uri hardcodate din o bază care nu mai există.
 - **C5** — constante moarte în `ExceptionConstants` , rămase după ștergerea excepțiilor: `COURSE_ID_NOT_FOUND` , `STUDENT_ID_NOT_FOUND` , `TEXT_ARGUMENT_NUMBER_INVALID` , `BOOK_ALREADY_EXISTS` . Plus `EMAIL_NOT_FOUND = "Email not found "` — două spații la final, care ajung în corpul răspunsului.
 - **C6** — `findByEmail` (`StudentRepository:51`) și `getByEmail` (`:26`) sunt aceeași interogare derivată cu două nume, folosite în două metode diferite ale aceluiași service. Păstrează `findByEmail` .
 - **C7** — `@Transactional` redundant pe metode, sub un `@Transactional` de clasă: `StudentCommandServiceImpl:28` , `:52` , `:62` · `BookCommandServiceImpl:34` .
 - **C8** — `StudentUpdateRequest:19` : `@NotNull` pe un `int` nu se poate declanșa — un primitiv nu e null. Ce te apare e `@Positive` de dedesubt (runda 5, C5).
 - **C9** — `Student.java:75-81` : constructorul `Student(String text)` care parsează CSV, rămas de la `Runner` . Nimic din API nu-l cheamă (runda 5, C6).
 - **C10** — `BookQueryServiceImpl.java` se termină fără linie nouă, cu trei rânduri goale înainte de acoladă (runda 5, C7).
 - **C11** — `springdoc-openapi-starter-webmvc-ui` are versiune fixată în `pom.xml` (`3.1.0`), deși restul dependențelor o iau din `spring-boot-starter-parent` (runda 5, C8).
 - **C12** — colecția Postman arată tot portul `8094` și 28 de cereri, dar cele cinci cereri de la B1 lipsesc. Adaugă un folder `conflicte/` cu ele, înainte de a repara B1 — ca să vezi cele cinci 500-uri cu ochii tăi, apoi cele cinci 409-uri după fix.
-

Before / After (doar pentru 🚫)

#	Acum	Corect
B1	un singur <code>@ExceptionHandler</code> de conflict, cu <code>EmailAlreadyUsed</code>	<code>handleConflict</code> cu toate 4: <code>+CourseNameAlreadyInUse,</code> <code>+StudentAlreadyEnrolledInThisCourse,</code> <code>+BookAlreadyAssignedToThisStudent</code>
B2	<code>PUT /courses/{id}</code> nume ocupat → 500 (verificarea rulează, excepția n-are handler)	409 cu <code>"message":"Course name already in use"</code> — se închide odată cu B1
B3	<code>if(findAllByAgeLessThan(age).isEmpty())</code> <code>throw</code> . <code>if(findAllStudentsWithBooks().isEmpty())</code> <code>throw</code>	<code>return</code> <code>studentRepository.findAllByAgeLessThan(age);</code> — lista goală e <code>200 []</code>
B4	<code>reset(); seed();</code> la fiecare pornire	<code>@Profile("seed")</code> pe <code>Runner</code>

Q&A — de răspuns înainte de runda 7

1. **B2.** La runda 5, `PUT /api/courses/{id}` cu numele altui curs dădea **409**; după ce ai scris verificarea pe care ți-am cerut-o, dă **500**. Explică în cuvinte de unde venea 409-ul de la runda 5 — și de ce verificarea ta, deși e corectă, l-a făcut să dispară.
2. **B1 + B3.** În ambele constatări ai aplicat regula exact acolo unde o cerere din Postman trecea, și nu mai departe: ai completat lista de not-found (unde aveai 4 cereri de `GET /{id}`), nu pe cea de conflict (unde aveai o singură cerere, pe studenți); ai curățat 14 metode de listare (cele 4 expuse plus cele vecine), nu pe cele 2 pe care nu le atinge nimic. Scrie ce ai putea verifica **înainte** de a zice „am terminat constatarea” — o comandă sau o listă, nu o intenție — care ți-ar arăta locurile unde regula nu a ajuns.
3. **M2.** `CourseQueryServiceImpl:67` e `return`
`Optional.ofNullable(courseRepository.findById(id).orElseThrow(NoCourseFound::new));` . Explică de ce `ofNullable` nu poate primi niciodată `null` aici, și ce se întâmplă în `EnrollmentCommandServiceImpl:76` cu `.orElseThrow(NoCourseFound::new)` de la capătul apelului.

Runda 5 — 2026-09-30 — sha `bda6ec9`

- ☒ B1 advice-ul nu-și cunoștea jumătate din excepții, 20 de locuri pe 500 — **jumătatea „not found” închisă** (runda 6, B1 pentru jumătatea „conflict”, 5 locuri)
- ☒ B2 REGRESIE `PUT /api/students/{id}` nu mai putea salva nimic — **închis**,
`findByEmail(...).filter(...).ifPresent(...)`

- B3 `GET /api/enrollments/{id}` inexistent → 500 prin NPE (`getById` derived query întoarce `null`) — **închis**, 404
- B4 listele goale → 500, a treia rundă, 16 locuri — **închis pe tot ce e expus** (14/16; runda 6, B3 pentru cele 2 rămase)
- B5 `orElseThrow()` fără argument, 7 locuri — **închis complet**
- B6 verificarea de duplicat la carte întreba despre mulțimi diferite — **închis**, `existsByStudentIdAndName`
- B7 `PUT` pe o înscriere cu aceeași pereche → 500, a treia rundă — **închis**, 200
- B8 fiecare pornire șterge tot ce a scris API-ul — **NEATINS**, a treia rundă (runda 6, B4)
- B9 la update pe curs regula de unicitate nu se executa niciodată — **verificarea e scrisă și rulează**, dar statusul a regresat de la 409 la 500 (runda 6, B2)
- M1 `NoCourseFound` vs `CourseIdNotFound` , excepții duble — **închis**, 4 clase șterse
- M2 `findByDepartmentOrderByNameAsc` nu sorta — **închis**
- M3 interogarea rulează de două ori, 12 locuri — **parțial**, 5 rămase (runda 6, M1)
- M4 `Optional` care aruncă, `orElseThrow` mort la apelant — **NEATINS** (runda 6, M2)
- M5 `POST` → 200, nu 201 — **NEATINS** (runda 6, M3)
- M6 `DELETE /api/enrollments` ia id-urile din corp — **NEATINS** (runda 6, M4)
- M7 `PUT /api/books/{id}` cere și suprascrie `createdAt` — **NEATINS** (runda 6, M5)
- M8 ștergerea unui curs șterge în tăcere înscrierile — **NEATINS** (runda 6, M6)
- M9 19 din 27 de metode de query neexpuse — **NEATINS** (runda 6, M7)
- M10 structura cerută de patru runde — **NEATINS**; `web/` a devenit `sistem/` , nume românesc în cod englezesc (runda 6, M8)
- M11 trei convenții de nume în DTO-urile de înscriere — **NEATINS** (runda 6, M9)
- M12 două adnotări `@Transactional` diferite — **închis**, toate pe cea Spring
- M13 `@PathVariable` importat într-o interfață de service — **închis**
- C1 importuri nefolosite — **parțial**, de la 22/19 fișiere la 14/11 (runda 6, C1)
- C2 `deletebook` / `updatebook` · C3 `getCourseById` în `EnrollmentController` · C4 `Runner` 718 → 671 linii · C5 `@NotNull` pe `int` · C6 constructor CSV în `Student` · C7 fără linie nouă la final · C8 springdoc cu versiune fixată — **toate NEATINSE**

Runda 4 — 2026-09-25 — sha `8ecffc9`

- B1 `unique = true` pe ambele `@JoinColumn` → aplicația nu pornește pe bază nouă — **închis** (probat pe bază ștearsă, pornire în 2,1s)
- B2 `Runner` șterge tot la fiecare pornire — **NEATINS** (runda 5, B8)
- B3 niciun `@RestControllerAdvice` , 12 situații → 500 — **închis pe jumătate**: advice-ul există, 14 din 20 de situații răspund corect, 20 de `throw` rămân pe 500 (runda 5, B1)
- B4 duplicatul de carte întreabă despre mulțimi diferite — **NEATINS** (runda 5, B6)
- B5 `orElseThrow()` gol în 7 locuri — **NEATINS** (runda 5, B5)

- 🚫 B6 unicitate lipsă la update pe student și pe curs — „reparat” cu bug nou: pe student a intrat `existsByEmail` și a blocat orice `PUT` (runda 5, B2); pe curs, neatins (runda 5, B9)
- 🚫 B7 `PUT` pe aceeași pereche de înscriere → 500 — **NEATINS**, a treia rundă (runda 5, B7)
- 🚫 B8 listele goale → 500, 13 locuri — **NEATINS**, iar 5 verificări moarte au fost activate în loc de șterse (runda 5, B4)
- ✅ M1 `@Transactional` lipsă pe command service-uri — **închis** (toate patru)
- 🟡 M2 `findByDepartmentOrderByNameAsc` nu sortează — **NEATINS** (runda 5, M2)
- 🟡 M3 query dublu, 12 locuri — **NEATINS** (runda 5, M3)
- 🟡 M4 `== null` imposibil în `EnrollmentCommandServiceImpl` — **NEATINS** (runda 5, M4)
- 🟡 M5 `POST` → 200 în loc de 201, 4 locuri — **NEATINS** (runda 5, M5)
- 🟡 M6 `DELETE /api/enrollments` cu corp — **NEATINS** (runda 5, M6)
- 🟡 M7 `createdAt` editabil la `PUT /api/books` — **NEATINS** (runda 5, M7)
- 🟡 M8 ștergerea cursului șterge înscrierile în tăcere — **NEATINS**, reprodus din nou (runda 5, M8)
- 🟡 M9 niciun `GET /{id}` — **închis**: toate patru resursele + `search?email=` (runda 5, M9 pentru restul metodelor neexpuse)
- 🟡 M10 structura (`ExceptionConstants` ×4, `services/interfaces/`, `repository`) — **NEATINSĂ**, a patra rundă (runda 5, M10)
- 🟡 M11 trei convenții de nume în JSON-ul de enrollment — **NEATINS** (runda 5, M11)
- ✅ C4 constructor cu repository neatribuiri • ✅ C5 `@Component` pe `BookQueryServiceImpl` • ✅ C1, C2 importuri nefolosite semnalate punctual — **închise**
- 🟢 C3, C6, C7, C8, C9, C10 — **neatinse** (runda 5, C1–C6)

Runda 3 — 2026-09-23 — sha 62915d4

- ✅ B4 `@Valid` fără reguli pe `EnrollmentUpdate/DeleteRequest` — **închis** (`PUT {}` → 400)
- ✅ B5 `createBook` nu putea reuși niciodată — **închis** (`POST /api/books` → 200)
- ✅ B6 trei command service-uri inaccesibile din API — **închis** (CRUD pe toate patru)
- 🚫 B1 liste goale → 500 — **NEATINS**, 13 locuri (runda 4, B8)
- 🚫 B2 niciun `@RestControllerAdvice` — **NEATINS**, acum 12 situații (runda 4, B3)
- 🚫 B3 `PUT` pe aceeași pereche → 500 — **NEATINS** (runda 4, B7)
- ✅ M6 `Enrollment.equals` crăpa fără student/curs — **închis**
- ✅ M7 `deleteStudent` ignora emailul — **închis** (semnătura trecută pe `UUID`)
- 🟡 M2 query dublu — **închis** în `CourseQueryService`, rămas în celelalte (runda 4, M3)
- 🟡 M8 unicitate lipsă la update — **NEATINS**, acum expus prin HTTP (runda 4, B6)
- 🟡 M1 graf cu două colecții • M3 `== null` imposibil • M4 `POST` → 200 • M5 `DELETE` cu corp • M9 `@Transactional` lipsă • M10 structura — **toate NEATINSE** (runda 4, M1–M10)
- 🟢 C1, C2, C3, C4, C5, C7 — **neatinse** (runda 4, C1–C6)

Runda 2 — 2026-08-19 — sha 8b84a6d

-  B1 `Course.enrollments` neinițializat → NPE — **închis**
-  B2 `StudentSummary.getId()` `Long` peste `UUID` — **închis**
-  B3 `@EntityGraph` către un graf inexistent — **închis**
-  B4 `createBook` nu poate reuși niciodată — **NEATINS** (runda 3, B5)
-  B5 `updateEnrollment` duplică perechea — verificarea a fost adăugată, dar prinde și rândul propriu (runda 3, B3); constrângerea `UNIQUE` pe tabelă tot lipsește
-  B6 `Enrollment.equals` crapă fără student/curs — **NEATINS** (runda 3, M6)
-  B7 `deleteStudent` ignoră emailul din request — **NEATINS** (runda 3, M7)
- M1 `updateCourse` scrie orbește · M2 email duplicat la update · M3 excepție greșită · M4 query dublu · M5 `== null` imposibil · M6 `@Transactional` lipsă · M7 factory · M8 `Enrollment` incomplet — **toate NEATINSE** (runda 3, M2–M9)
- C1–C8 — neatins, cu excepția celor dispărute odată cu refacerea fișierelor

Runda 1 — 2026-08-05

-  `spring-boot-starter-web` + `controllers/` — **închise la runda 3**
-  `@RestControllerAdvice` — **NEATINS de trei runde** (runda 3, B2)
-  `services/interfaces/` → `services/` + `services/impl/` — **NEATINS**
-  `ExceptionConstants` duplicat (acum în 4 pachete) — **NEATINS**
-  plural inconsistent `repository` vs `models` / `dtos` / `services` — **NEATINS**
- Decizia de arhitectură: `Student` = aggregate root, `book/` fără `services/` proprii — contrazisă de `BookCommandService`, de reluat explicit

